

תסריט דמו – מערכת AI לטריאז' נכסים

פרויקט גמר · יהודה רוקח · אורך יעד: 9 דקות · בכל סצנה: מה על המסך · מה לומר · ולמה זה עובד ככה

הכנה לפני הקלטה

- לפתוח מראש את כל הטאבים: WebUI על ה-Elastic IP בפורט 5050 (/ Submit / Dashboard Assistant), n8n בפורט 5678, ו-AWS Console (Bedrock, DynamoDB, S3, EC2, Secrets Manager, Systems Manager).
- לפתוח ב-IDE את הקבצים שנצטרך להראות: n8n workflow, train.py, main.py + label_condition.py של ה-Image Analyser, rag_service/main.py + prompts.py, guardrails_service/main.py + prompts.py, webui/server.py + app.js + system_prompts.py + prompt_log.md, docker-compose.yml, deploy/ec2-userdata.sh, aws_utils.py + gemini_utils.py.
- להכין מראש listing תקין לדוגמה (תיאור + 2-3 תמונות אמיתיות כולל אחת של מטבח), תמונת כלב/-off listing לבדיקת ה-not_a_room, טקסט spam/off-topic לבדיקת ה-input guardrail, ו-listing עם טענה לא-אמיתית (מחיר מומצא / certification פיקטיבי) לבדיקת ה-output guardrail.
- לפתוח טרמינל SSM-session פתוח אל ה-instance עם nvidia-smi מוכן להרצה, ולוודא שכל שמונת הקונטיינרים רצים (docker compose ps).
- לוודא שה-Knowledge Base (3KTFERDLUV) ו-Bedrock, ושטבלאות pt_listings ו-pt_events פתוחות ב-DynamoDB.
- לנקות את חלון הציאת לפני ההקלטה כדי שתרחיש ה-V8 follow-up יתחיל נקי.
- לבדוק רמת קול, רזולוציית מסך (להגדיל פונט ב-IDE), ולסגור התראות/חלונות מיותרים.

מסך

- להציג את ה-WebUI החי דרך ה-Elastic IP בפורט 5050 – שלושת הטאבים: Submit, Dashboard, Assistant.
- לצייר/להראות דיאגרמה קצרה: ח8n <- WebUI -> ארבעת ה-, image:8002, rag:8001 microservices + Bedrock + Ollama (agent:8000, guardrails:8003).
- להצביע על שני שירותי ה-Bedrock (Knowledge Bases + Guardrails) ועל Gemini ככל ה-generation.

קריינות (לקרוא)

שלום, אני אציג את ה-AI-Powered Real Estate Property Triage System – מערכת שמקבלת מודעת נכס, תיאור חופשי ותמונות, ובאופן אוטומטי בודקת אותה, מחלצת ממנה את העובדות, מנתחת את התמונות, כותבת brief מקצועי לסוכן הנדל"ן, ומנתבת אותה לצוות הנכון – residential או commercial. במילים אחרות, היא עושה triage לנכסים: סינון, העשרה וניתוב, בלי שאדם צריך לקרוא כל מודעה ידנית. כל הסטאק רץ על instance בודד אחד ב-EC2 עם GPU, ועולה בפקודה אחת של docker compose. בלב המערכת יש ח8n שמתזמר שמונה שלבים, ומתחתיו ארבעה microservices ב-FastAPI: RAG ל-Image Analyser, comparables, לתמונות, Guardrails לבטיחות וסוכן רב-שלבי. בחרתי בדיוק שני שירותי Knowledge Bases – AWS Bedrock ו-Guardrails – וכל שאר ה-generation מאוחד למודל אחד של Gemini. בדקות הקרובות אני אעבור על כל זה מקצה לקצה: נגיש מודעה אמיתית, נראה את ה-pipeline פועל, נראה את הדשבורד, את שתי שכבות ה-guardrails, את הציאט המקורקע, ואת מה שמאחורי הקלעים.

נקודות טכניות – מה בנינו ולמה

- הקלט: description חופשי + images. הפלט: brief ב-routing + Markdown ל-residential/commercial.
- מתזמר: (8 nodes) ח8n. מתחתיו 4 microservices ב-, image:8002, rag:8001 FastAPI: guardrails:8003, agent:8000.
- שני שירותי Bedrock בדיוק: (RAG) Knowledge Bases ו-Guardrails (בטיחות). כל ה-generation ב-Gemini (gemini-2.5-flash).
- הכל על instance יחיד ב-EC2 g4dn.xlarge עם GPU, מורם דרך Docker Compose.

הגשת מודעה תקינה מקצה-לקצה – ה-pipeline ב-n8n + ה-Image Analyser

20% n8n · שירותים 25% (Image Analyser + Agent) · grounding/anti-hallucination

3:35-0:55

מסך

- ב-Submit tab: למלא תיאור נכס אמיתי, להעלות 2-3 תמונות (כולל מטבח) וללחוץ Submit.
- לעבור ל-canvas n8n ולהראות את שמונת ה-nodes משמאל לימין; להצביע על הקו -> Webhook Guardrails Input -> IF passed
- ללחוץ Guardrails Input ולהראות URL host.docker.internal:8003/check/input ו-jsonBody ששולח רק {text: description}.
- ללחוץ Information Extractor ולהראות את ה-attributes ואת ה-extract ONLY system prompt 'facts... never infer, guess, or invent'.
- ללחוץ AI Agent ולהראות את שלושת הכלים מחוברים מלמטה (rag_lookup, image_analyser, property_agent) ואת ה-Gemini Chat Model המשותף.
- לפתוח train.py: לסמן את MultiHeadNet, את הלולאה requires_grad=False (שורות 82-83), ואת room_head/cond_head; לפתוח classes.json ולהראות 7 מחלקות כולל not_a_room.
- להריץ דמו חי של POST /analyse עם תמונת מטבח -> {room_type, condition_score, confidence}; להראות שה-condition מגיע מ-gemini_condition (Gemini Vision) וה-CNN הוא fallback.
- ללחוץ LLM Chain ולהראות את ה-Routing rule וה-Output Parser עם enum ['residential', 'commercial']; להראות את ה-brief שנרנדר ב-Submit tab.

קריינות (לקרוא)

נתחיל בלב העניין – נגיש מודעה אמיתית ונעקוב אחריה מקצה לקצה. הזנתי תיאור, העליתי כמה תמונות, ולחצתי Submit. עכשיו בואו נראה מה קורה ב-n8n. הכל מתחיל ב-Webhook שמקבל POST על הנתבי listing עם התיאור והתמונות. הדבר הראשון שאני עושה, עוד לפני שום קריאת LLM, זה Guardrails Input – אני שולח רק את התיאור לשירות ה-Guardrails, והוא מחזיר pass או fail. שמתי את זה ראשון בכוונה כי זה fail-fast: ה-Information Extractor, ה-AI Agent עם שלושת הכלים וה-LLM Chain כולם עולים כסף וזמן, אז את הסינון הזול אני עושה קודם – ככה מודעת spam נדחית מיד ב-node IF בלי לבזבז אפילו טוקן אחד. רק אם זו מודעה אמיתית אני ממשיך. אחר כך בא ה-Information Extractor שמחלץ שישה שדות מבניים מהטקסט – סוג נכס, מיקום, מחיר, מספר חדרים, certifications ו-features – והוא מורשה לחלץ רק מה שכתוב מפורש, אסור לו לנחש או להמציא, כי השדות האלה מזינים את כל ההמשך כולל החלטת הניתוב. עכשיו הלב: ה-AI Agent. במקום לקרוא לשירותים בצורה קשיחה, נתתי לו שלושה כלים והוא בוחר בעצמו מה להריץ – rag_lookup – comparables, image_analyser לתמונות, ו-property_agent לשאלות רב-שלביות. למה Agent ולא קריאות hardcoded? כי מודעה בלי תמונות לא צריכה את ה-image_analyser, ושאלה פשוטה לא צריכה את הסוכן הרב-שלבי – ה-Agent מחליט לפי תיאור הכלים, וזה הרבה יותר חכם מ-branching ידני. נעצור רגע על ה-Image Analyser עצמו, כי כאן יש החלטת הנדסה מעניינת. בניתי רשת אחת חכמה ולא שתיים: EfficientNet-Bo עם משקלי ImageNet, והקפאתי לגמרי את הגב – בקוד אתם רואים את הלולאה שמגדירה requires_grad ל-False. למה? כי הדאטהסט קטן, והגב כבר יודע לזהות

צורות ומרקמים מ-ImageNet, אז אני מאמן רק שתי שכבות ליניאריות זולות מעל אותו גב - מהיר ויציב, בלי overfitting. יש שני ראשים: ראש סוג-חדר וראש מצב. שימו לב למחלקה השביעית, not_a_room - מחלקת דחייה שהוספתי בכוונה, כי רשתות CNN נוטות לבטחון-יתר על תמונות מחוץ להתפלגות; ככה תמונת כלב או מסמך מסומנת not_a_room במקום להיכפות לחדר. ויש גם סף ביטחון של 0.55 - מתחתיו אני מחזיר uncertain, כי עדיף שהמערכת תגיד 'לא בטוח' מאשר תטעה בביטחון. הגעתי ל-84.4% דיוק על שבע המחלקות, הרבה מעל סף ה-75%. ועכשיו הנקודה הכנה: ראש המצב אומן על תוויות שבוטסטרפתי עם Gemini Vision, אבל כשבדקתי אותו על חדרים רעי-מצב-אך-מסודרים הוא טעה. אז קיבלתי החלטת serving היברידית: סוג החדר מגיע מה-CNN שלי, אבל ציון המצב מוגש בזמן inference על ידי Gemini Vision עצמו עם אותה רובריקה - וה-CNN נשאר כ-fallback אם Gemini נכשל. אחרי שה-Agent אסף את הניתוחים, ה-LLM Chain כותב את ה-brief ב-Markdown ובאותה קריאה בוחר routing לפי חוק מפורש: דירה, בית או וילה זה residential; משרד, מסחר או מחסן זה commercial. Output Parser עם enum מבטיח שהערך תמיד תקין, וה-grounding נכפה בכל שכבה - אסור להמציא מחירים, אסור להוסיף טענות שיווק, ואסור להשוות לנכסים אלא אם ה-analyst אמר זאת מפורשות. וזה ה-brief המוכן.

נקודות טכניות - מה בנינו ולמה

- סדר ה-8 Guardrails -> (POST /listing, responseMode=responseNode) Webhook nodes: Input -> IF passed1 -> Information Extractor -> AI Agent -> LLM Chain -> Guardrails Output -> Router (Switch)
- Guardrails Input שולח רק {text: description} ל-host.docker.internal:8003/check/input; fail--ל-fast לפני כל קריאת LLM.
- Information Extractor (Gemini) מחלץ 6 שדות: property_type, location, price, num_rooms, key_features, certifications - facts בלבד, never infer/guess/invent.
- AI Agent (Gemini) בוחר מבין 3 כלי image_analyser, rag_lookup (:8001/query), HTTP: (8002/analyse: - חדר + 1-5 condition לכל תמונה), property_agent (:8000/agent/run). מעוגן בתוצאות הכלים.
- Image Analyser: MultiHeadNet מעל EfficientNet-Bo קפוא (requires_grad=False), שני ראשים room_head (7 מחלקות) + not_a_room (5); cond_head היא reject-class; סף 0.55 -> 'uncertain'.
- דיוק סוג חדר 84.4% ולידציה / 84.6% תמונות טריות - מעל סף 75%.
- Serving היברידי: room מה-condition, CNN מ-Gemini Vision (אותה רובריקת 1-5); cnn_condition הוא condition_score=null -> not_a_room.fallback.
- LLM Chain (chainLlm, hasOutputParser=true) כותב Output routing; brief_markdown + Gemini Chat Model Parser enforce enum ['residential', 'commercial']. אחד משותף ל-3 ה-nodes.

מסך

- לפתוח את Dashboard tab - להראות שורות אירועים (exec_ms, avg_condition, routing, guardrail flags) כולל ההגשה החדשה.
- ללחוץ על שורה כדי לפתוח detail modal ולהראות שהתמונות נטענות מה-URL הקבוע.
- ב-server.py: לסמן את ddb_put_doc_ - להדגיש id / ts / doc ואת doc: json.dumps(doc) שעוקף את ה-Decimal; ולסמן את public_url (URL ישיר בלי 'never expire', 'presigning').
- ב-docker-compose.yml: להראות, pt_listings, DDB_LISTINGS_TABLE.
- pt_events, UPLOAD_BUCKET, DDB_EVENTS_TABLE.
- ב-AWS Console: DynamoDB - שתי הטבלאות עם Item שבו שדה doc הוא - S3 JSON string; prefix uploads / ציבורי (תמונה נטענת בדפדפן) מול /listings / פרטי (Access Denied).

קריינות (לקרוא)

ההגשה שזה עתה עברה מופיעה כבר בדשבורד. כל שורה היא אירוע אנליטיקה: כמה זמן לקח, ציון המצב הממוצע, ה-routing, והאם עבר את ה-guardrails. אני אלחץ על שורה כדי לפתוח את ה-detail view, ושימו לב שהתמונות נטענות - אגיע לזה עוד רגע. אבל קודם, סיפור אמיתי שלמדתי ממנו. בהתחלה שמרתי את הליסטינגים והאירועים בקובץ בתוך Docker volume - ואז עשיתי rebuild לקונטיינר, ה-volume נמחק, ואיבדתי את כל ההגשות. מהרגע ההוא עברתי ל-DynamoDB בתור system-of-record עמיד: הוא שורד rebuilds, הוא pay-per-request אז בנפח הזה הוא עולה בערך אפס. יש לי שתי טבלאות - pt_listings מחזיקה את הרשומה המלאה כולל מפתחות התמונות וה-brief, ו-pt_events מחזיקה שורות אנליטיקה רזות לדשבורד. ויש כאן טריק חשוב: אני שומר כל רשומה כ-JSON document יחיד, עם שלושה שדות - id, ts ו-doc. למה? כי DynamoDB לא יודע לאחסן float נייטיב - הוא דורש Decimal לכל מספר, וה-condition scores וה-exec_ms שלי הם floats. במקום להילחם בכל ההמרה, אני עושה json.dumps לכל הרשומה למחרוזת אחת - וזה עוקף את כל בעיית ה-Decimal, והקריאה חזרה היא json.loads פשוט. ואם הטבלאות לא מוגדרות, נופלים אחורה ל-JSONL מקומי, כך שאפשר לרוץ לוקאלית ולהריץ את כל הטסטים בלי שום credentials של AWS. עכשיו לתמונות, וזה מתחבר למה שראיתם בדשבורד: כל תמונה נוחתת ב-S3 תחת prefix בשם uploads / שהוא public-read, וזה נותן URL קבוע שלעולם לא פג. למה זה קריטי? כי presigned URLs פגים אחרי כשעה, וה-URL הזה נשמר בתוך רשומות ה-DynamoDB ובתוך טקסט ה-KB - לינק שפג היה מרקיב בתוך הרשומות והתמונות בדשבורד היו נשברות. ובכוונה רק uploads / ציבורי; טקסט ה-KB תחת prefix נפרד בשם /listings / נשאר פרטי לגמרי, כי זה דאטה פנימי לחיפוש שאסור לחשוף.

נקודות טכניות - מה בנינו ולמה

- שתי טבלאות DynamoDB: pt_listings (רשומה מלאה לצורך pin + detail view בציאט) ו-pt_events (אנליטיקה רזה לדשבורד, כולל נדחות).
- ddb_put_doc_ כותב Item עם doc (S) / ts (S) / id (S=json.dumps(S) של כל הרשומה) - עוקף את דרישת ה-Decimal של DynamoDB ל-floats.

- קריאה: `ddb_all_docs_` עושה `scan` (נפח נמוך) + מיון לפי `ts; fallback` ל-JSONL מקומי כשהטבלה לא מוגדרת - פיתוח/טסטים בלי AWS.
- המעבר ל-DynamoDB נובע מאובדן דאטה אמיתי: Docker volume נמחק ב-`rebuild`.
- תמונות ב-S3 תחת `public_url`; `public-read`; `uploads/` בונה URL ישיר בלי `never` -> `presigning` `expire. listings` (טקסט KB) נשאר פרטי.

מסך

- ב-guardsrails_service/main.py: להראות שני ה-endpoints /check/input (שורה 88) ו-/check/output (שורה 115) ואת מבנה התשובה {pass, reason, safe_text}.
- לסמן את apply_guardrail_ (שורות 63-80): runtime.apply_guardrail_ עם source='INPUT', איסוף סיבות מ-contentPolicy/topicPolicy/wordPolicy, והשורה blocked = (fail-closed) .action=='GUARDRAIL_INTERVENED'.
- ב-prompts.py: להראות את language + is_property_listing INPUT_CLASSIFIER_PROMPT (he/en/other) ולהסביר allow-list ולא denylist.
- להריץ /check/input POST עם מודעה תקינה בעברית (pass=true), ואז עם spam/off-topic ועם טקסט בשפה אחרת - להראות דחייה והודעת REJECT_MESSAGES הדו-לשונית.
- להראות זאת גם דרך ה-WebUI: הגשת spam שנדחת מיד.

קריינות (לקרוא)

עכשיו שכבת ההגנה - שירות ה-Guardrails, שהוא גם השירות השני שלי שמבוסס על Bedrock. יש שני endpoints: אחד לקלט ואחד לפלט, ושניהם מחזירים pass, reason ו-safe_text. הרעיון המרכזי: כל בדיקה רצה בשתי שכבות, ואני רוצה להסביר למה דווקא שתיים. השכבה הראשונה היא Bedrock guardrail - ApplyGuardrail מנוהל שנותן safety גנרי מהקופסה: סינון שנאה, אלימות, תוכן מיני, עלבונות, prompt-attacks, profanity ו-denied topics. אני קורא לו עם source שמסמן INPUT או OUTPUT, ואוסף את סיבות החסימה משלושה סוגי policy. אבל יש כאן מגבלה עקרונית: denied-topics זה denylist - הוא יודע להגיד מה אסור, אבל לא יכול לבטא מה מותר. וזה בדיוק התפקיד של השכבה השנייה, ה-rail של Gemini. בקלט, Gemini מריץ classifier שהוא allow-list: הוא שואל 'האם זו באמת מודעת נכס אמיתית, ובשפה שאנחנו תומכים בה?'. אי אפשר לכתוב denied-topic שאומר 'דחה כל דבר שהוא לא נדל"ן' - צריך מודל שמבין כוונה. ובאותה נשימה אני פותר גם את הדחייה הרב-לשונית: אם השפה היא לא עברית ולא אנגלית, אני מחזיר הודעת דחייה מנומסת ודו-לשונית. בואו נראה את זה חי: מודעה תקינה בעברית מקבלת pass=true; spam או טקסט off-topic מקבל דחייה עם בקשה לתאר סוג, מיקום, גודל, חדרים ומחיר; וטקסט בשפה לא נתמכת מקבל את ההודעה הדו-לשונית. ודבר אחרון וחשוב: המערכת fail-closed - כל intervention של Bedrock חוסם, וגם כל כשל של ה-classifier מחזיר pass=False. בשירות בטיחות אני תמיד מעדיף לדחות הגשה תקינה מאשר להעביר פעם אחת תוכן לא בטוח.

נקודות טכניות - מה בנינו ולמה

- שני endpoints: POST /check/output ו-POST /check/input, מחזירים {pass, reason, safe_text}.
- שתי שכבות: Layer 1 = Bedrock ApplyGuardrail (Bedrock service #2); Layer 2 = rail של Gemini.
- apply_guardrail_ אוסף סיבות מ-contentPolicy.filters (שנאה/אלימות/מיני/עלבונות/prompt-attacks), wordPolicy.managedWordLists, topicPolicy.topics.

- allow-list: is_property_listing=true הוא INPUT_CLASSIFIER_PROMPT (temperature=0.0) רק למודעה אמיתית; denied-topics language he/en/other. לא יכול לבטא 'קבל רק נדל"ן'.
- not is_property_listing -> language דו-לשוני; != he/en -> REJECT_MESSAGES['other']
REJECT_MESSAGES['not_listing']
- classifier -> fail-closed: GUARDRAIL_INTERVENED חוסם גם על policy שלא מנינו; כל כשל -> .pass=False

מסך

- ב-prompts.py: להראות את OUTPUT_AUDITOR_PROMPT (שורות 31-56) עם שלושת סוגי ההפרה INVENTED_FACT / GUARANTEE / LEGAL_CLAIM.
- ב-check_output (שורות 128-140): להדגיש ש-Layer 2 (factuality) רץ רק אם source קיים – Bedrock grounding כ-op-no כשאין מקור, ולכן Gemini הוא ה-gate הראשי.
- ב-n8n: לחוץ Guardrails Output ולהראות שהוא שולח text=brief_markdown ו- source=description+ממצאי ה-AI Agent; להצביע על Router -> IF output passed או Respond Review.
- להריץ POST /check/output עם דוח שמכיל מחיר מומצא / certification פיקטיבי מול source נתון – להראות violation מסוג 'review': status=pass=false, INVENTED_FACT.

קריינות (לקרוא)

השכבה השנייה היא בדיקת הפלט, וכאן הסכנה שונה לגמרי. ה-input שאל 'האם זו מודעה אמיתית?'. ה-output שואל שאלה אחרת – 'האם ה-brief שה-LLM כתב אמין מול המקור?'. כשמודל מייצר דוח נדל"ן, הסכנה האמיתית היא הזיות: מחיר שהומצא, certification שלא קיים, או טענה משפטית כמו 'ללא שעבודים' שאין לה כיסוי במקור. ב-n8n אתם רואים ש-Guardrails Output שולח את ה-brief כ-text ואת התיאור המקורי וממצאי ה-Agent כ-source, ושירות ה-Guardrails מריץ את ה-auditor של Gemini שמשווה את הדוח למקור. כל הפרה – INVENTED_FACT, GUARANTEE או LEGAL_CLAIM – מחזירה pass=false ומפנה את הדוח ל-human-review במקום לשחרר אותו אוטומטית. שימו לב לנקודה עדינה: ה-grounding של Bedrock כאן הוא רק best-effort ומשני – בדיקת ה-factuality רצה רק אם קיים source, אז כשאין מקור היא פשוט op-no, ולכן ה-Gemini auditor הוא ה-gate הראשי. בואו נראה את זה: אני שולח דוח עם מחיר מומצא מול ה-source האמיתי, וה-auditor מסמן violation מסוג INVENTED_FACT, מחזיר pass=false, וב-flow זה מנתב ל-Respond Review עם status review וגם כאן fail-closed – כל כשל של ה-auditor מחזיר pass=False, כי עדיף להעביר דוח לבדיקה אנושית מאשר לשחרר עובדה מומצאת.

נקודות טכניות – מה בנינו ולמה

- ב-check_output: קודם Bedrock על הדוח, ואז OUTPUT_AUDITOR_PROMPT של Gemini – אבל רק אם source קיים.
- סוגי violation: INVENTED_FACT / GUARANTEE / LEGAL_CLAIM; כל -> human-review -> pass=False.
- ב-Guardrails Output n8n: שולח text=brief_markdown ו-source=description+ממצאי ה-AI Agent; Router -> IF output passed (true) או Respond Review status='review' (false).
- Bedrock grounding כאן best-effort/משני (op-no כשאין source); Gemini auditor הוא ה-gate הראשי.
- fail-closed: כל exception ב-pass=False -> auditor.

- ה-grounding כבר נכפה upstream: ה-Agent וה-LLM Chain מחויבים לעגן ולא להמציא מספרים/טענות שיווק – ה-output guardrail הוא רשת הביטחון האחרונה.

V8 follow-up-ה- Ollama Assistant chat על GPU, מקורקע ל-KB, תמונות, ותיקון

Chat/Surface #5 25% · grounding · GPU

8:10~6:55

מסך

- ב-Assistant tab: תור 1 'anything in Beersheba?' - להראות תשובה שמצטטת listing לפי ID (SUBMITTED-... או ###) ועם תמונות שנטענות מתחת לבועה.
- תור 2 'tell me about it' - להראות שהתשובה ממשיכה על אותו listing בדיוק בלי סתירה (זה ה-pin של V8).
- ב-server.py: לסמן OLLAMA_MODEL=llama3.1 (שורה 38) ו-open_ollama_stream; לסמן retrieval_query_ (users[-3:], שורה ~294) ו-pinned_listing_blocks_ עם ddb_get_doc_ (שורה ~314); ולסמן את טקסט ה-context שאוסר position number ודורש ID (שורה ~371).
- ב-app.js: (שורות 93-116): לסמן את attachListingPhotos עם אותו LISTING_ID_RE כמו בשורת fetch ל- /api/listing/<id/>; ב-DevTools Network להראות שה-URL הוא .../s3.amazonaws.com/uploads קבוע.
- לפתוח prompt_log.md בכניסות V7/V8 של Surface 5 ולקרוא את תרחיש 'ca9876' שתועד חי.

קריינות (לקרוא)

עכשיו לב הציאת. מאחורי הקלעים רץ אצלי Llama 3.1 דרך Ollama על ה-GPU - מודל קוד-פתוח מקומי, בלי תלות בספק חיצוני. אבל מודל בגודל 8B נוטה להמציא - מחירים, כתובות, אפילו לינקים מזויפים - אז לא נתתי לו לדבר מהבטן. הוא מקורקע על ה-Knowledge Base ב-Bedrock דרך שירות ה-RAG במצב retrieve-only: ה-KB רק מחזיר את ה-listings הרלוונטיים כטקסט, וה-Llama מנסה רק מהם, אז הוא עונה על הקורפוס האמיתי שלי ולא על הזיות. וכל אחד ב-prompt אוסר עליו להמציא פרט שלא נמצא בהם. עכשיו התיקון הכי מעניין, V8. בגרסה הקודמת השליפה רצה רק על ההודעה האחרונה - אז כששאלתי 'משהו בבאר שבע?' וקיבלתי listing מסוים, וכתבתי אחר כך 'ספר לי עליו', שלוש המילים האלה בלי נושא שלפניו סט אחר לגמרי, והמודל ענה 'אין לי כזה listing' - בסתירה לתור הקודם. אז עשיתי שני דברים. אחת: אני בונה את שאילתת השליפה משלושת תורות המשתמש האחרונים, כך ש-follow-up מעורפל עדיין נושא את הנושא. ושתיים, וזה ה-pinning: כל listing שכבר הוזכר בשיחה אני סורק מההיסטוריה לפי ה-id, שולף אותו ישירות מ-DynamoDB - לא מהשליפה - ומצמיד אותו ראשון ל-context, כך ש-listing שכבר דיברנו עליו לעולם לא נעלם. ולמה אני מתייחס ל-listings לפי ID יציב כמו L015 ולא לפי '1, 2, 3 Listing'? כי הסט הנשלף משתנה בין תור לתור, אז מספר מיקום זז וגורם לסתירה עצמית - ID יציב לנצח. תראו את זה חי: תור ראשון מחזיר listing עם ID, ותור שני 'tell me about it' ממשיך על אותו אחד בדיוק. ולסיום, התמונות מתחת לתשובה: הצד-לקוח סורק את טקסט התשובה עם בדיוק אותו regex של ה-id-ים, ולכל id עושה fetch ל-/api/listing, שמושך את הרשומה מ-DynamoDB ומחזיר את ה-URL הציבורי הקבוע מ-S3 שלא פג אף פעם - אותו URL מהסיפור של S3 קודם. ו-seed listings בלי תמונות פשוט מחזירים 404 ולא מציגים כלום, fire-and-forget.

נקודות טכניות - מה בנינו ולמה

- Ollama + llama3.1:latest על ה-GPU; פותח סטרימינג ל-/api/chat; 502 נקי אם Ollama לא זמין.

- with_insight=False עם RAG_URL/query POST שולח GROUNDED: rag_listings_context KB - (retrieve-only) מחזיר listings בלבד, Llama מנסח מהם.
- follow--query - #1 V8 מחבר את users[:3] (שלושת תורות המשתמש האחרונים) כך ש--follow קט מעורפל נושא נושא.
- id - #2 V8 PINNING: _pinned_listing_blocks סורק היסטוריה עם LISTING_ID_RE, שולף לפי id מ-DynamoDB עם _ddb_get_doc, מוכנס ראשון ל-context (blocks.setdefault).
- הפניה לפי ID יציב (SUBMITTED / L###-...) ולא מספר מיקום - הסט הנשלף משתנה בין תורות; תועד חי תרחיש 'ca9876' ב-V8->V7 prompt_log.
- תמונות: attachListingPhotos (app.js) משתמש באותו regex כמו השרת, fetch ל-/<id>/api/listing api_listing מחזיר public_url קבוע מ-fire-and-forget, 404 -> seed listings /S3 uploads.
- ניסוי aya-expanse:8b לעברית טובה יותר אבל התעלם מ-directive השפה בסירובים -> חזרה ל- llama3.1 + אכיפת שפה בקוד.

מאחורי הקלעים וסיכום – Bedrock, Gemini 2 בכל מקום, IAM/Secrets בלי מפתחות, ו- deploy על GPU

אדריכלות/10% Gemini + Bedrock 2 • 15% deploy • אבטחה

9:00-8:10

מסך

- ב-docker-compose.yml: להצביע על שמונת ה-services ועל בלוק ollama (driver: nvidia, count: all, capabilities: [gpu]); להראות שמועבר רק GEMINI_SECRET_NAME ולא הערך, וכן KB_ID 3KTFERDLUV ו-GUARDRAIL_ID huksxm9z68f6.
- בטרמינל SSM: להריץ nvidia-smi - לראות את ה-Tesla T4 ואת תהליך ollama; ולהראות שהחיבור הוא Session Manager בלי SSH.
- ב-aws_utils.py: get_gemini_api_key עם lru_cache; ב-gemini_utils.py: client()/generate_credentials מול Google - אימות מול Google דרך credentials של AWS בלבד.
- ב-AWS Console: EC2 (g4dn.xlarge + Elastic IP + IAM role), Secrets Manager (property: triage/gemini-api-key בלי הערך), Bedrock (KB 3KTFERDLUV + Guardrail huksxm9z68f6) - בדיוק שניים).
- לפתוח את deploy/ec2-userdata.sh ולעבור על השלבים: git clone, Compose plugin, docker+git, cp aws s3 של --build -d docker compose up model.pth.

קריינות (לקרוא)

נסגור עם מה שמאחורי הקלעים. כל המערכת - ארבעת ה-microservices, ה-n8n, ה-Ollama וה-WebUI - רצה על instance בודד אחד ב-EC2 מסוג g4dn.xlarge עם GPU מסוג Tesla T4, ועולה בפקודה אחת. למה GPU? כי הציאט על Ollama לקח בערך 57 שניות ב-CPU - לא שמיש; ברגע שהעברתי אותו ל-T4 זה ירד ל-2 עד 3 שניות, וזה מה שהופך אותו לחוויית זמן-אמת. בקובץ ה-compose אתם רואים את הבלוק שמוסר את ה-GPU ל-Ollama - driver nvidia, count all, capabilities gpu - ועל המכונה התקנתי את הדרייבר במצב open-kernel-module דרך DKMS, ככה הוא שורד עדכוני קרנל, וגם את ה-nvidia-container-toolkit. עכשיו הנקודה שאני הכי גאה בה: אין אף מפתח על המכונה. ל-instance יש IAM role, ו-boto3 שואב credentials מה-default chain. את מפתח ה-Gemini אני לא שומר בשום קובץ - בקוד, get_gemini_api_key קורא בזמן ריצה ל-Secrets Manager, ו-lru_cache מחזיק אותו פעם אחת לכל תהליך; ב-compose אני מעביר רק את שם ה-secret, אף פעם לא את הערך. הניהול כולו דרך AWS Systems Manager - אין לי בכלל מפתח SSH, ופורט 22 סגור. ולגבי Bedrock: הדרישה היא לפחות שניים, ובחירתי בדיוק שניים - Knowledge Bases מאחורי ה-RAG, ו-Guardrails מאחורי הבטיחות. במקום להוסיף תלות Bedrock שלישית ל-generation, איחדתי את כל ה-generation - ה-RAG insight, הסיווג והאודיט ב-guardrails, ה-agent, ציון המצב בתמונות וצמתי ה-n8n - למודל אחד של Gemini, gemini-2.5, flash, שאני כבר שולט בו: model אחד, footprint מינימלי ב-Bedrock, ושימוש חוזר בדפוסים שעובדים. ה-deploy עצמו הוא bootstrap אחד ב-boot - מתקין docker ו-git, מושך את model.pth מ-S3, ומריץ docker compose up. וזהו: מערכת triage מלאה לנכסים, מקצה לקצה, מקורקעת ובטוחה. תודה.

נקודות טכניות – מה בנינו ולמה

- instance יחיד: 8; EC2 g4dn.xlarge + NVIDIA Tesla T4, us-east-1, Amazon Linux 2023; services ב-docker-compose (rag/image/guardrails/agent/n8n/ollama/webui).
- GPU passthrough: deploy.resources.reservations.devices (driver: nvidia, count: all, capabilities: [gpu]); דרייבר open-kernel-module דרך Chat: DKMS + nvidia-container-toolkit. CPU ~57s -> ~2-3s על T4.
- אין מפתחות על המכונה: get_gemini_api_key + IAM role קורא Secrets Manager בזמן ריצה עם lru_cache; ב-compose מועבר רק GEMINI_SECRET_NAME (property-triage/gemini-api-key).
- ניהול דרך SSM Session Manager – אין SSH, פורט 22 סגור; גישה ציבורית דרך Elastic IP; security group חושף רק 5050 (WebUI) ו-5678 (n8n).
- בדיוק 2 שירותי huksxm9z68f6 GUARDRAIL_ID + (RAG) 3KTFERDLUV KB_ID Bedrock: generation ב-Gemini gemini-2.5-flash ב-DRAFT (Guardrails) version.
- RAG service: Bedrock KB מסוג VECTOR על S3 Vectors (נבחר על פני OpenSearch Serverless בגלל עלות ~\$5-6/יום), Titan Embed v2 (1024 ממדים, cosine), insight; top-3 מעוגן עם ציטוט retrieve-only ID, כשלא צריך insight; הטמעה חוזרת של הגשות מאושרות ל-KB.
- boot: docker+git, Compose plugin, git clone, aws s3 cp ב-deploy/ec2-userdata.sh; restart: unless-stopped; docker compose up --build -d; model.pth.

טיפים להקלטה

- שמרו על קצב דיבור רגוע – 7 הסצנות ביחד הן כ-9 דקות, אז אל תמהרו; אם חורגים, סצנה 7 היא המקום לקצר.
- הקליטו את הקריינות והמסך בנפרד או עם דאבל-טייק – קל יותר לסנכרן הקלקות עם משפטים ארוכים.
- כשמראים קוד, הגדילו את הפונט והדגישו את השורות המדויקות (highlight) במקום לגלול הרבה – הצופה צריך לראות את requires_grad=False, את json.dumps, doc: את users[-3:] ואת pinned_listing_blocks_ ברגע שמדברים עליהם.
- הריצו את ההגשה התקינה (סצנה 2) פעם אחת לפני ההקלטה כדי שה-execution ב-n8n כבר יהיה ירוק וה-brief ירונדר מהר במצלמה.
- לתרחיש ה-V8 (סצנה 6) – נקו את הציאט קודם והשתמשו בדיוק בשני התורות 'anything in Beersheba? ואז 'tell me about it' כדי שה-pin יודגם בלי רעש.
- הכינו מראש את ה-payloads ל-/check/output, /check/input ו-/analyse (curl או Postman) כדי שהדמואים החיים בסצנות 4-5 וב-Image Analyser ירוצו בלחיצה אחת.
- בסצנה 7 – אם nvidia-smi או חיבור ה-SSM איטיים, צרבו צילום מסך מוכן כ-fallback כדי לא לתקוע את ההקלטה.
- חזרו פעם אחת על המעברים בין הטאבים (AWS -> Assistant -> Dashboard -> n8n -> Submit Console) כדי שהזרימה תהיה חלקה; כל מעבר אמור להתחבר למשפט הראשון של הסצנה הבאה.

- הזכירו את אחוז הרובריקה בראש כל סצנה בראש שלכם (לא בקול) כדי לוודא כיסוי מלא: ח8, שירותים, guardrails, chat, אחסון, אדריכלות ו-Bedrock-2 - כולם מכוסים.